

Implementing Threads in User Space

Definition

User-level thread implementation means that the entire threads package is managed in user space by a library. The kernel does not know that the threads exist and treats the whole process as a normal single-threaded process.

1. Where Threads Can Be Implemented

Main Choices

Threads can be implemented in:

- user space,
- the kernel,
- or a hybrid of both.

This Section

This section focuses only on **user-space threads**, where a run-time library performs all thread management.

2. Basic Idea of User-Level Threads

Kernel View

When threads are implemented entirely in user space, the kernel knows nothing about them. From the kernel's point of view, it is simply managing ordinary processes.

How It Is Built

The threads package is implemented by a **library** plus a **run-time system**. This run-time system provides operations such as:

- `pthread_create`,
- `pthread_exit`,
- `pthread_join`,
- `pthread_yield`,
- and usually several others.

Major Immediate Benefit

User-level threads can be used even on operating systems that do not support threads in the kernel.

3. Run-Time System and Thread Table

Private Thread Table

Each process must maintain its own private **thread table**. This table is managed by the run-time system, not by the kernel.

What the Thread Table Stores

The thread table is similar to a process table, but only for per-thread state such as:

- program counter,
- stack pointer,
- registers,
- thread state,
- and related execution information.

How It Is Used

When a thread becomes **ready** or **blocked**, the information needed to restart it later is saved in the thread table, just as the kernel saves process information in a process table.

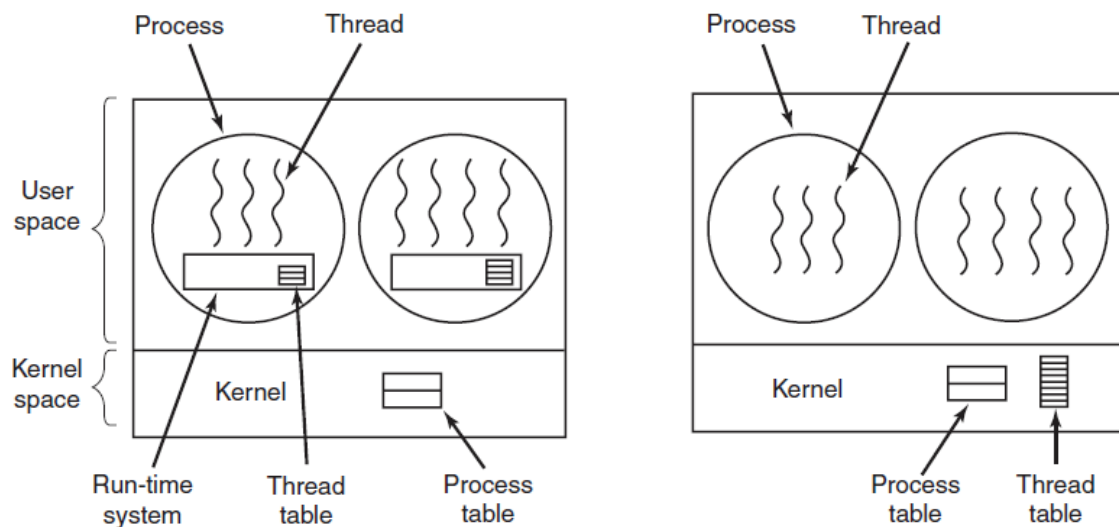


Figure 1: A user-level threads package and a kernel-managed threads package.

4. How Thread Switching Works

Local Blocking

If a thread performs an operation that blocks only with respect to its own process, such as waiting for another thread in the same process, it calls a run-time system routine.

Switch Sequence

The run-time system then:

1. checks whether the current thread must block,
2. saves its registers in the thread table,
3. selects another ready thread,
4. restores that thread's saved registers,
5. switches the stack pointer and program counter.

Why It Is Fast

This thread switch happens entirely in user space. No trap to the kernel is needed, no process-level context switch is needed, and there is usually no cache flush. As a result, user-level thread switching is often at least an order of magnitude faster than switching through the kernel.

5. Advantages of User-Level Threads

Advantage	Explanation
Works on existing OSes	Can be implemented even if the OS kernel has no native thread support
Very fast switching	Switching threads is just a library-level operation inside the process
Fast scheduling	Scheduler and save/restore logic are local procedures, not kernel calls
Custom scheduling	Each process can use its own scheduling policy
Better scalability	No per-thread kernel table and kernel stack cost for huge thread counts

Custom Scheduling

Different applications may need different scheduling behavior. With user-level threads, each process can choose its own thread scheduling algorithm.

Garbage Collector Example

In some applications, such as those using a garbage-collector thread, it is useful that the programmer can control thread scheduling without the kernel interrupting at inconvenient moments.

6. Problem 1: Blocking System Calls

Most Serious Practical Problem

If a user-level thread makes a blocking system call, the kernel blocks the entire process because the kernel does not know that other runnable threads exist inside it.

Example

Suppose one thread reads from the keyboard before any input is available. If the thread actually enters the blocking system call, the whole process stops, including all other threads.

Why This Is Bad

One main reason for using threads is to allow one blocked activity not to stop the rest of the program. User-level threads fail badly here when blocking system calls are involved.

7. Possible Workarounds

7.1 Nonblocking System Calls

Theoretical Fix

One solution is to change system calls so that they become nonblocking. For example, a **read** could return immediately with zero bytes if no input is ready.

Why This Is Unattractive

This approach requires changing the operating system and also changes the meaning of familiar calls like **read**, which would force many user programs to change as well.

7.2 Wrapper or Jacket

Wrapper Idea

If it is possible to check in advance whether a system call would block, the thread library can place a **wrapper** (also called a **jacket**) around that call.

UNIX Example with **select**

On many UNIX systems, the library can call **select** first to see whether a **read** is safe. If the read would block, the library does not make the real system call and instead runs another thread.

Weakness

This method works, but it is inefficient, awkward, and requires rewriting parts of the system call library.

8. Problem 2: Page Faults

Kernel Does Not Know About Threads

If one user-level thread causes a page fault, the kernel blocks the entire process while bringing the missing page in from disk.

Why This Hurts

Even if other threads in that process are ready to run, they cannot run because the kernel only sees one process and blocks it as a whole.

9. Problem 3: No Automatic Preemption Inside the Process

Running Forever

In a pure user-level thread system, if one thread keeps running and never voluntarily enters the run-time system, no other thread in that process will run.

Why This Happens

There are no per-thread clock interrupts inside the process to force round-robin scheduling. The scheduler only runs when a thread explicitly gives it a chance.

Possible but Messy Fix

The run-time system could request periodic clock signals to regain control, but this is crude, messy, and can introduce significant overhead or interfere with the application's own use of timers.

10. Most Devastating Criticism

Blocking-Heavy Applications

Programmers often want threads most in applications where blocking happens frequently, such as multithreaded Web servers. But these are exactly the cases where pure user-level threads perform badly because they make many system calls.

Why Kernel Help Becomes Natural

Once a thread traps into the kernel for a system call, it is usually not much more work for the kernel to switch to another thread if the old one blocks. That makes kernel-supported threads much more practical for blocking-intensive workloads.

CPU-Bound Programs

For programs that are almost entirely CPU bound and rarely block, threads often provide little advantage anyway. In such cases, the motivation for using threads is much weaker.

11. User-Level Switching Sketch

Conceptual Flow

```
thread_yield()
{
    save_current_thread_state();
    next = choose_ready_thread();
    load_thread_state(next);
    run(next);
}
```

Meaning

This shows the main attraction of user-level threads: saving state, scheduling, and restoring another thread can all happen without entering the kernel.

12. Main Conclusion

Takeaway

User-level threads are fast, flexible, and easy to add to systems without kernel thread support. However, they suffer from major problems with blocking system calls, page faults, and lack of automatic preemption, which makes them poorly suited for many real blocking-intensive applications.

13. Quick Review

Exam-Focused Points

- User-level threads are implemented entirely in user space by a library and run-time system.
- The kernel does not know the threads exist.
- Each process keeps its own private thread table.
- Thread switching in user space is very fast because it avoids kernel traps and full context switches.
- User-level threads allow custom scheduling per process.
- A blocking system call made by one thread can block the entire process.
- Page faults caused by one thread also block the whole process.
- Without voluntary yielding or timer tricks, one thread can monopolize the CPU inside the process.
- Wrappers or jackets can reduce blocking-call problems, but they are awkward and inefficient.
- User-level threads are least suitable for applications that block often, such as Web servers.